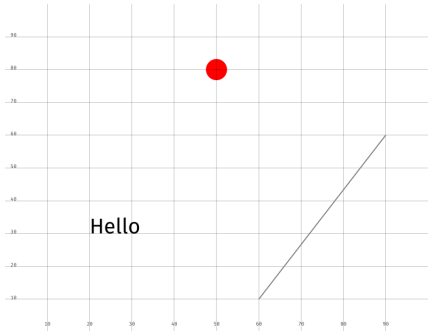


The decksh Tutorial



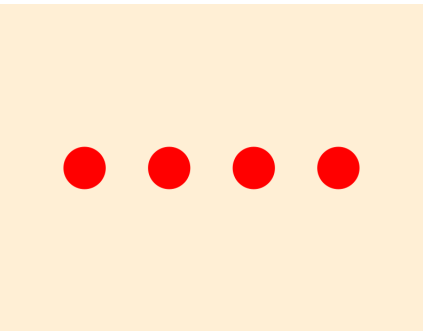
Your First Deck



The Coordinate System

hello **hello**

Variables

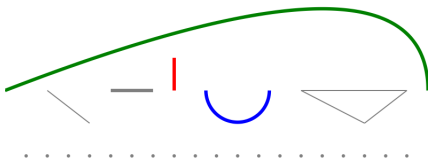


Colors

Beginning
Centered
Ending
Rotate 20°

We need
all hands
on deck

Working with Text



Stroked Graphics



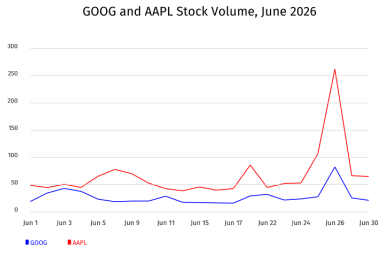
Filled Shapes



Images

- First
- First
1. First
- First
- Second
- Second
2. Second
- Second
- Third and Last
- Third and Last
3. Third and Last
- Third and Last

Lists



Charts



Geographic Functions

Your First Deck

This is your first deck. Congratulations.

A deck is usually contained in a single text file, conventionally with the suffix ".dsh". Each line in a deck begins with a keyword: a single word that reflects its function. Comments begin with '/' and may be placed at the beginning, or anywhere on a line of decksh code.

The code for the deck is contained between the 'deck' and 'edek' keywords. The 'slide' and 'eslide' keywords define the boundaries of a slide. You may include an arbitrary number of slides in a single deck.

Keywords are followed by items, known as 'arguments', separated by spaces or tabs, that define the keyword's behavior. In the example, the 'slide' keyword has two arguments: "black" and "white", which means the slide background is black and its text is white. If you omit the optional arguments, the default of black text on a white background is applied.

The 'ctext' keyword means 'centered text'. Its arguments are the text ("hello, world"), followed by the coordinates of the text, and followed by the size of the text. The image is placed at the bottom of the slide, scaled to span the slide's width.



```
// First
deck
  slide "black" "white"
    ctext "hello, world" 50 50 15 // say it loud
    image "earth.jpg" 50 0 100 0 // image at the bottom
  eslide
edek
```

Think of a deck as a series of pages; the building blocks of your story.

The Coordinate System

Decksh uses the traditional Cartesian coordinate system to place elements on the canvas. The x, y coordinates represent percentages ranging from 0-100; the coordinates are passed as arguments as a pair of numbers, x first, then y.

The origin (0,0), is at the lower-left, with the x coordinate increasing from left to right, with the y coordinate increasing bottom to top. The middle of the canvas is (50,50): half-way from the left and half-way up from the bottom.

For example, to place the text "Hello" left-aligned at 20% from the left, and 30% up (20,30), and the text size at 5% of the canvas width, use:

```
text "Hello" 20 30 5
```

Other elements like lines and circles may also be placed. For example

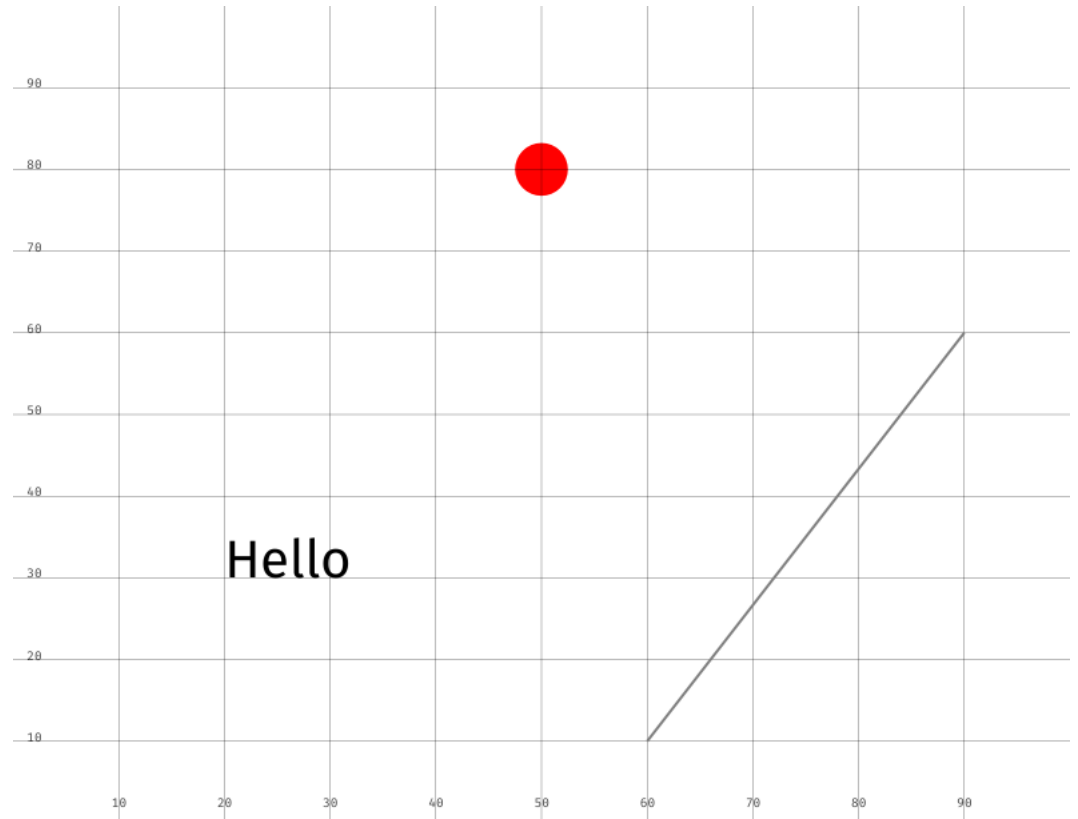
```
circle 50 80 5 "red"
```

places a red circle at (50,80), the size of the circle is 5% of the canvas width.

To draw a line from (60,10) to (90,60)

```
line 60 10 90 60
```

Finally, note the `ruler` keyword, which makes a convenient scale.



```
deck
  slide
    ruler 10
    text "Hello" 20 30 5
    circle 50 80 5 "red"
    line 60 10 90 60
  eslide
edeck
```

Place anything, anywhere with precision and consistency.

Variables and Assignments

Variables provide a method for attaching a name to an object so that the name may be used for other operations.

Variables may refer to either numbers, or text strings (data defined between " characters). Use the '=' after the name to assign the value of variable. The general syntax is:

name = value

In the example, (1) the value of "hello" is assigned to s, (2) 5 and 50 to the variables x and y and (3) ts is assigned the value of 5. Next you can use the variables:

text s x y ts

which is equivalent to

text "hello" 5 50 5

When using variables that contain numbers, you can also perform binary operations: add (+), subtract (-), multiply (*), divide (/), and modulo (%). For example,

a = x + y // a gets the sum of x and y

You can also use 'assignment operations' on a single variable, for example to add 10 to an existing variable x, use:

x *= 10

Use variables as the levers that control the deck's elements and their relationships.

hello

hello

```
deck
  slide
    s="hello"
    x=5
    y=50
    ts=5
    text s x y ts
    x=x+30
    ts+=15
    text s x y ts
  eslide
edeck
```

Using Color

Many decksh elements use color as an optional argument.

The color may be specified in four ways:

(1) A named color as defined in the SVG standard; these are colors like "red" and "blue", but also include fairly exotic names like "papayawhip".

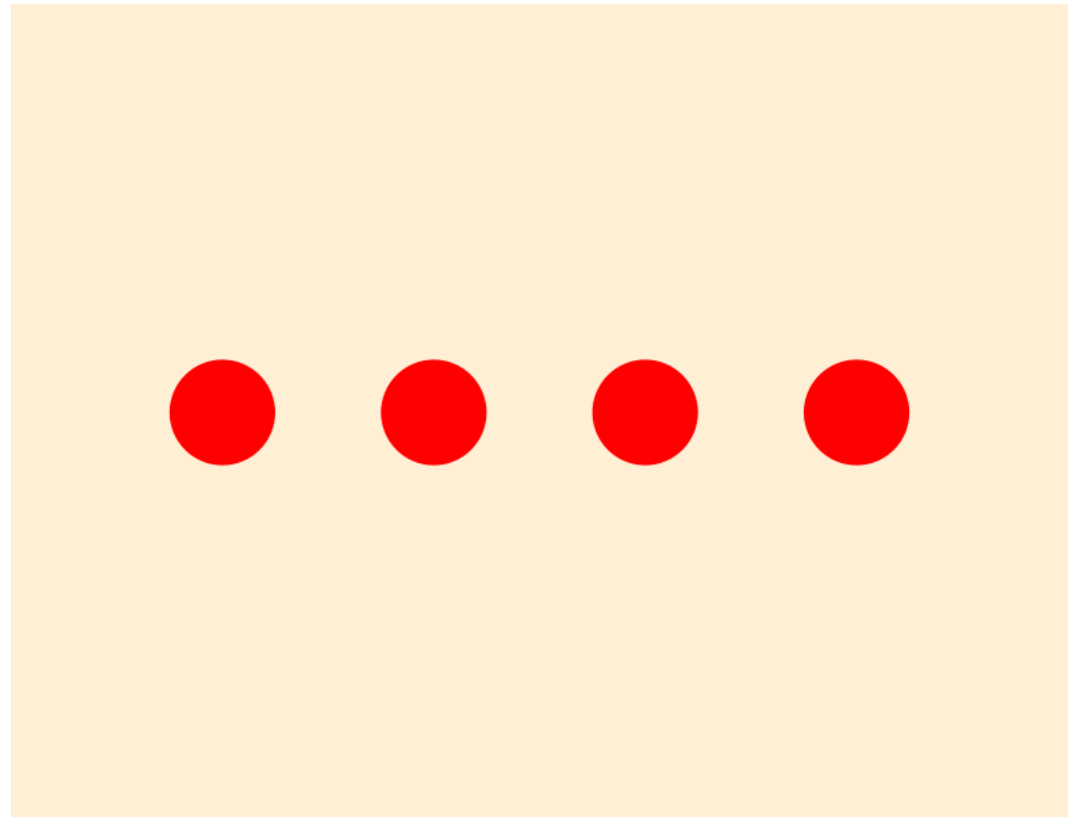
(2) A RGB triple, which uses numbers ranging from 0-255 to refer to amount of red, green, and blue in the color. For example, "rgb(0,0,0)" is black, "rgb(255,255,255)" is white, with "rgb(128,128,128)" as gray. By the way, papayawhip is "rgb(255,239,213)".

(3) The hex RGB format, in the form of "#rrggbb", where rr, gg, and bb are the red, green and blue components of the color as hexadecimal digits ranging from "00" (zero) to "ff" (255). Papayawhip is "#ffefd5" in hex.

(4) The HSV (hue, saturation, value) format uses three numbers: the first, hue, ranges from 0-360; the saturation and value (brightness) range from 0-100.

For example, "hsv(0,100,100)" is a fully saturated red, and "hsv(120,50,50)" green with diminished saturation and brightness.

Use color as a method of communicating meaning, regardless of how you name it.



```
deck
  slide "papayawhip"
    c1="red"           // named color
    c2="rgb(255,0,0)"  // rgb
    c3="#ff0000"       // hex rgb
    c4="hsv(0,100,100)" // hue, saturation, value
    circle 20 50 10 c1
    circle 40 50 10 c2
    circle 60 50 10 c3
    circle 80 50 10 c4
  eslide
edeck
```

Working with Text

Text is an important decksh element and may be used in various ways. Most text keywords use this pattern:

keyword "text" x y size [font] [color] [op]

Text may be aligned (begin, end, and centered) relative to a specific point. The corresponding keywords are 'text' (and its synonym 'btext') 'etext' and 'ctext'. The point is specified by the x and y coordinate, the size of the text is the third 'size' argument (a percentage relative to the width of the canvas)

All text keywords have an optional set of arguments that define the font (either "sans", "serif", "mono", or "symbol"), the color of the text, and its opacity (ranging from 0 (invisible) to 100 (solid color)) If no optional arguments are specified the default values are "sans", "black", and 100.

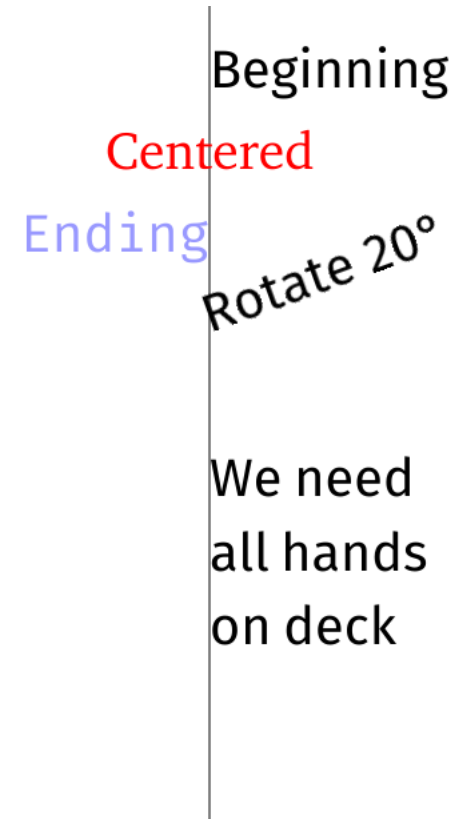
Rotated text ('rtext') is specified like the others, except you must include the angle of rotation (0-360).

rtext "text" x y angle size [font] [color] [op]

A block of text is also similar, but the width of the block must be specified.

textblock "text" x y width size [font] [color] [op]

Build your story using text.



```
deck
  slide
    tx=50
    ts=5
    saying="We need all hands on deck"
    line          tx 0 tx 100
    btext "Beginning" tx 90 ts "sans"
    ctext "Centered" tx 80 ts "serif" "red"
    etext "Ending" tx 70 ts "mono" "blue" 40
    rtext "Rotate 20°" tx 60 20 ts
    textblock saying tx 40 20 ts
  eslide
edeck
```

Stroked Graphics

decksh supports stroked graphic shapes including lines, polylines, arcs, and curves.

The general syntax includes the arguments for the shape, with optional arguments for line width, color and opacity; (defaulting to 0.2%, "gray" and 100%)

To draw a line between (10,50) and (20,70):

```
line 10 50 20 70
```

Dotted lines are similar, but you may specify the gap between dots after the line width:

```
dline 10 50 90 50 0.2 5
```

Horizontal and vertical lines specify the beginning point and the length:

```
hline 10 50 30 and vline 10 50 30
```

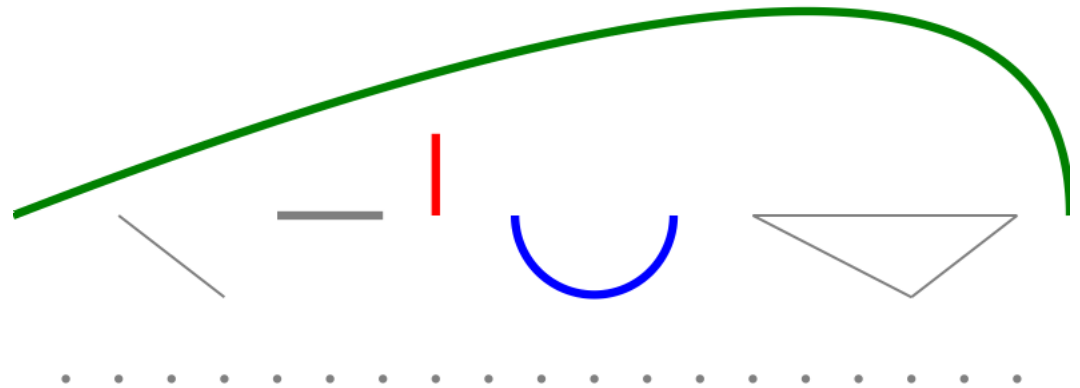
For arcs, specify the center of the arc, its width and height, and start and ending angles.

```
arc 50 50 10 10 0 90
```

Polygons use two string arguments: the first are list of x coordinates, and the second are the y coordinates.

Quadratic Bezier curves take three pairs of coordinates: the beginning point, the control point, and the end point.

You have the draw the line somewhere.



```
deck
  slide
    base=50
    lw=0.8
    line    10 base 20 40
    hline   25 base 10 lw
    vline   40 base 10 lw "red"
    arc     55 base 15 15 180 360 lw "blue"
    polyline "70 85 95" "base 40 base"
    curve   0 base 100 100 100 base lw "green"
    dline   5 30 95 30 lw 5
  eslide
edeck
```

Filled Shapes

decksh supports filled shapes including circles, ellipsis, square, rectangle, rounded rectangle, star, and polygon.

Filled shapes are specified by location arguments (x and y coordinates usually at the center of the shape), followed by dimensions (width and height), with optional arguments for color and opacity; if optional arguments are omitted, the color is "gray" with 100% opacity.

Circles and squares take the center point and a single dimension (width). Ellipses and rectangles also take a center point with a width and height.

Rounded rectangles are specified like regular rectangles, but they require you to specify the radius of the corners.

Filled polygons are specified like polylines: two string arguments that specify the sets of x and y coordinates.

To make stars, specify the center of the star, the number of points, and the inner and outer radii.



```
deck
  slide
    base=50
    circle 10 base 5
    ellipse 20 base 10 5 "red"
    square 30 base 5 "green"
    rect 40 base 10 15 "blue"
    rrect 55 base 10 15 5 "orange"
    star 55 base 5 2 6 "blue"
    polygon "65 85 95" "base 60 base" "purple" 50
  eslide
edock
```

Use these elements to shape your slides.

Images

An image is the welcome companion to text when making decks.

To place an image, you specify its name (JPEG, PNG, and GIF image formats are supported), location, and dimensions. The general syntax is:

image "name" x y width height

The specified coordinate refers to the middle of the image, and the width and height refer to its dimensions (in pixels). An alternative (and recommended) syntax scales the image to the width of the canvas:

image "name" x y canvas-width 0

For example:

image "follow.jpg" 50 50 50 0

Places the JPEG image in the middle of the slide, scaled at 50% of the canvas width.

To add an image caption, use the 'cimage' keyword. The syntax is similar, but an additional caption argument is used:

cimage "cloudy.jpg" "Cloudy Day" 50 50 50 0



Head in the clouds

```
deck
  slide
    image "follow.jpg"                25 50 45 0
    cimage "cloudy.jpg" "Head in the clouds" 75 50 45 0
  eslide
edeck
```

The artful arrangement of pictures help illuminate your deck.

Lists

With decksh you can make many kinds of lists: plain, bulleted, numbered, centered.

The type of list is specified by the keyword: 'list' for plain, 'blist' for bulleted, 'nlist' for numbered, 'clist' for centered.

The arguments for lists include the x and y location and the text size of the list.

For example to begin a list at (20, 40) with text size 3 use:

```
list 20 40 3
```

Following the list type is any number of list items (li "item name"), one per line. The list ends with "elist" on a line by itself.

Decksh does not have native support for tables, but you can construct tables using "parallel lists", by aligning a series of plain lists to a set of columns.

Make a list and check it twice.

First	● First	1. First	First
Second	● Second	2. Second	Second
Third and Last	● Third and Last	3. Third and Last	Third and Last

```
deck
  slide
    ls=3
    ly=95
    list 0 ly ls
      li "First"
      li "Second"
      li "Third and Last"
    elist
    blist 25 ly ls
      li "First"
      li "Second"
      li "Third and Last"
    elist
    nlist 55 ly ls
      li "First"
      li "Second"
      li "Third and Last"
    elist
    clist 90 ly ls
      li "First"
      li "Second"
      li "Third and Last"
    elist
  eslide
edeck
```

Charts

decksh has built-in support for a variety of chart types with these corresponding keywords: barchart, linechart, areachart, scatterchart and dotchart.

To make a chart, specify the chart type and the input file. Data files contain a series of tab separated name,value pairs. For example, a line of data looks like: January<tab>100

Optional arguments control the chart data, label, and value colors. For example to make a line chart from data in "foo.d":

```
linechart "foo.d" [color] [label-color] [value-color]
```

Special variable names control the placement and other aspects of the chart. The variables 'chartTop', 'chartBottom', 'chartLeft', and 'chartRight' define the chart boundaries. If these are not set, the default values are: chartTop=80, chartBottom=30, chartLeft=10, and chartRight=90.

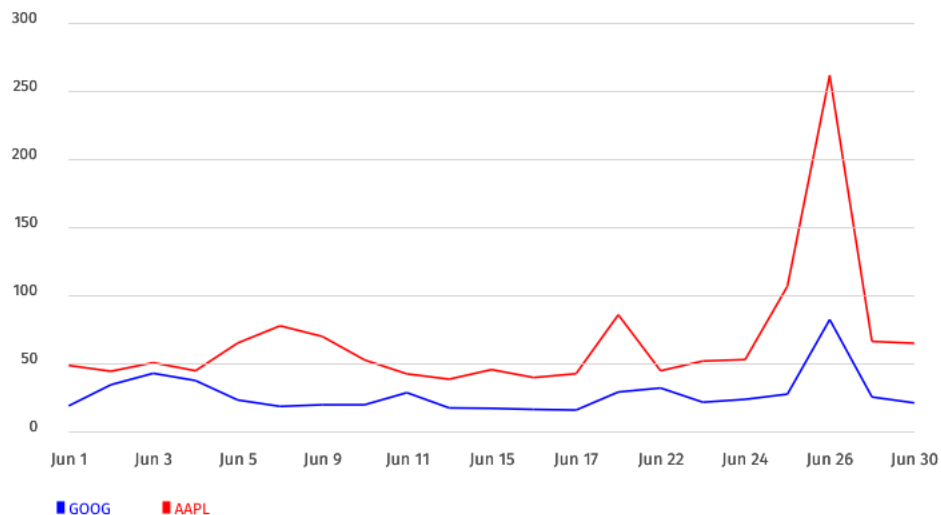
Other variables control items such as text size (chartTextSize), line width (chartLineWidth) and X-axis label interval (chartXLabel).

Chart legends are set with the 'legend' keyword with this syntax:

```
legend "text" x y size [font] [color]
```

Chart your course.

GOOG and AAPL Stock Volume, June 2026



```
deck
  slide
    ctext "GOOG and AAPL Stock Volume, June 2026" 50 90 3
    chartVal=0
    chartTextSize=2
    chartXLabel=2
    chartYRange="0,300,50"
    linechart "aapl-vol.d" "red"
    linechart "goog-vol.d" "blue"
    legend "GOOG" 10 20 1.5 "sans" "blue"
    legend "AAPL" 20 20 1.5 "sans" "red"
  eslide
edeck
```

Geographic Functions

decksh can read Shapefiles, KML, or geoJSON encoded data to produce geometric regions, borders and paths. Plain and labeled locations are also supported along with images at geographic locations.

The keywords for geo functions are 'geoborder' (geographic borders), 'georegion' (regions), 'geopath' (paths between points), various labels and markers ('geoloc', 'geolabel', 'geomark').

To place a map using GIS data in KML format:

```
georegion "africa.kml"
```

Geographic locations may be assigned to variables using geo URL syntax, followed by an optional <tab> and label. For example Los Angeles is defined as:

```
losangeles="geo:34.05,-118.25 Los Angeles"
```

The geographic bounding box of a map is defined by latitude (-90° to 90°) and longitude (-180° to 180°) in decimal degrees.

The 'geobound' keyword maps the specified lat/long boundaries to the slide canvas boundaries set with the 'geocanvas' keywords. The defaults for geobound are -90 90 -180 180, with geocanvas defaulting to 0 100 0 100 (geoXmin, geoXmax, geoYmin, geoYmax).



```
deck
  slide
    geobound -34.83 37.34 -17.52 51.266 // boundary of Africa
    geocanvas 10 90
    cairo="geo:30.04440,31.235700    Cairo"
    accra="geo:5.614818,-0.205874    Accra"
    geoloc    cairo "c" 4
    geoloc    accra "c" 4
    georegion "africa.kml" "tan"
    geopath   accra cairo 0.5 "red"
  eslide
edeck
```

A map shows the way.